

1. HTTP preserves its state by usually generating a session ID. This session ID generally contains information like user data, permissions and sometimes other contextual data. This ID is then “given” to the client, after which the client sort of “presents” this ID on subsequent requests. This ID needs to be very rigorous and very secure to prevent any attacks or hacking attempts. It is recommended that this ID contains 64 bits of entropy. Which is a fancy way of saying it needs 2^{64} possibilities, or 1.8446744×10^{19} . Then this session ID needs to be properly stored, generally it is stored with cookies or locally. Cookies are usually the recommended choice because websites have built in protections to prevent attackers from accessing the cookies. When a request is made, if the session ID is in a cookie, the cookie automatically gets sent with the request. But if its locally stored, the session ID must be contained inside the request when it is sent to the server.

2. The procedure to migrate from a Django database to a MariaDB database is as follows. First the database needs to be configured in settings.py. Ensure that the user has the permissions they need to perform later actions. Then you need to run the “python manage.py makemigrations” command then run the “python manage.py migrate” command. There are other handy commands involving migration with Django, there is sqlmigrate along with showmigrations. Sqlmigrate shows the SQL statements Django is using to migrate. And then showmigrations displays the current progress on the migration. You do not HAVE to use only MariaDB, Django supports systems like PostgreSQL, MySQL, and SQLite, with ups and downs to each system. As an aside, MySQL and MariaDB do not have full transactional DDL which means a rollback in the event of failure will have to be manually performed to fix it.